

CAP 4105 — Introduction to Machine Learning

Project 3: Unsupervised Learning

Topics: K-Means Clustering · Hierarchical Clustering · DBSCAN · Anomaly Detection · Cluster Evaluation

Project Overview

In Projects 1 and 2, you built **supervised** models — you had labeled data (pass/fail, exam scores) and trained models to predict those labels. In this project, you will explore a fundamentally different paradigm: **unsupervised learning**, where no labels are provided.

You will work with the same student performance features you already know (`hours_studied`, `sleep_hours`, `attendance_rate`, `prev_exam_score`), but this time the `passed` column has been removed. Your goal is to **discover hidden structure** in the data using clustering algorithms — and then evaluate whether the clusters you find correspond to any meaningful grouping.

At the end, you will be given a `ground_truth.csv` file containing the actual pass/fail labels, allowing you to compare your unsupervised discoveries with the true labels.

You will implement K-Means clustering **from scratch using NumPy**, then use **scikit-learn** to apply Hierarchical Clustering and DBSCAN, and finally compare all methods.

Logistics

Item	Detail
Team Size	1–3 students (individual or group)
Submission	Upload <code>.ipynb</code> file directly to Canvas
Deliverables	Jupyter Notebook (<code>.ipynb</code>) with all code, outputs, and analysis
Due Date	April 12, 2026, 11:59 PM

Important Submission Requirements

1. **Submit a `.ipynb` file** (not `.py` scripts or PDFs). This is the only accepted format.
2. **Your notebook must be fully executed** — all cells must show their outputs. A notebook with unexecuted or cleared cells will be treated as incomplete, and any task without visible output will receive zero credit.
3. **All plots must be displayed inline** within the notebook (not saved as separate image files).
4. **Follow the required notebook structure** (see Notebook Structure Requirement below). The grading system parses your notebook by section headers — missing or renamed headers may cause tasks to be missed during grading.
5. **Group submissions:** If you are working in a group, **every member must submit the same notebook individually on Canvas**. In addition:
 - Include a **Markdown cell at the very top** of your notebook listing all group members' names and Panther IDs.
 - Leave a **submission comment on Canvas** stating your group members (e.g., "Group: Alice Smith, Bob Lee, Carol Wang").

Why NumPy + scikit-learn?

Continuing the two-level approach from previous projects:

Level 1 — NumPy (from scratch): Implementing K-Means manually forces you to code the assign-update-repeat loop, compute centroids, and track WCSS. This ensures you understand the iterative optimization that K-Means performs.

Level 2 — scikit-learn (industry tool): Hierarchical Clustering and DBSCAN have more complex internal mechanics (dendrogram construction, density-based neighborhood expansion). Using scikit-learn lets you focus on **interpreting results** — reading dendrograms, tuning ϵ and MinPts, and comparing cluster quality — rather than implementing graph algorithms.

Note: Your NumPy K-Means and scikit-learn K-Means should produce similar (not necessarily identical) results. Small differences are expected due to initialization and convergence criteria.

Dataset

You are provided with two CSV files:

- `p3_students_unlabeled.csv` — Student data with **no labels** (~403 rows). Contains the features but no pass/fail information. This dataset may contain data quality issues.
- `p3_ground_truth.csv` — The actual pass/fail labels for the original 400 students. **Do not open or use this file until Part 6 (Cluster vs. Ground Truth Comparison)**. Using it earlier defeats the purpose of unsupervised learning.

Columns in `p3_students_unlabeled.csv`

Column	Description
<code>student_id</code>	Unique identifier
<code>hours_studied</code>	Hours spent studying (1-12)
<code>sleep_hours</code>	Average sleep hours per night (3-10)
<code>attendance_rate</code>	Lecture attendance percentage (40-100)
<code>prev_exam_score</code>	Previous exam score (30-95)
<code>lucky_number</code>	Self-reported number (irrelevant feature)

Notebook Structure Requirement

Your Jupyter Notebook **must** be organized with the following Markdown headers, in this exact order.

```
1 # 0. Imports and Setup
2 # 1. Data Preprocessing
3 ## 1.1 EDA and Data Cleaning
4 ## 1.2 Feature Selection and Normalization
5 # 2. K-Means Clustering
6 ## 2.1 NumPy Implementation (From Scratch)
7 ## 2.2 sklearn Verification
8 ## 2.3 Elbow Method
9 # 3. Hierarchical Clustering
10 ## 3.1 Agglomerative Clustering and Dendrogram
11 ## 3.2 Linkage Comparison
12 # 4. DBSCAN
13 ## 4.1 Core, Border, and Noise Points
```

```
14 ## 4.2 Parameter Tuning
15 # 5. Method Comparison
16 # 6. Cluster vs. Ground Truth Comparison
17 # 7. Summary & Reflection
```

Task Descriptions

Part 1: Data Preprocessing (10 points)

Task 1.1 — EDA and Data Cleaning (5 pts: C3 + Q2)

Completion (3 pts):

- Load `p3_students_unlabeled.csv` and display basic statistics (shape, dtypes, `.describe()`)
- Identify and handle missing values, duplicates, and impossible values
- Visualize the distribution of each feature (histograms or box plots)

Quality (2 pts):

- Justify your cleaning decisions (why drop vs. impute, etc.)
- Discuss: without labels, how does EDA differ from supervised learning? What patterns are you looking for?

Task 1.2 — Feature Selection and Normalization (5 pts: C3 + Q2)

Completion (3 pts):

- Decide which features to include for clustering (should `lucky_number` and `student_id` be included? Why or why not?)
- Apply **Z-score normalization** (standardization) to all selected features
- Show before/after statistics (mean, std)

Quality (2 pts):

- Explain why normalization is especially critical for distance-based clustering algorithms like K-Means and DBSCAN
 - Discuss what would happen if one feature has a much larger scale than others (e.g., `attendance_rate` ranges 40–100 while `sleep_hours` ranges 3–10)
-

Part 2: K-Means Clustering (25 points)

Task 2.1 — NumPy Implementation From Scratch (12 pts: C8 + Q4)

Implement the K-Means algorithm from scratch using NumPy. Your implementation should follow the 4-step algorithm from lecture:

1. Randomly initialize K centroids
2. **Assignment step:** Assign each data point to the nearest centroid (Euclidean distance)
3. **Update step:** Recalculate centroids as the mean of assigned points
4. **Repeat** until convergence or max iterations

Completion (8 pts):

- Implement a function `kmeans_numpy(X, k, max_iters=100, random_state=42)` that returns `(centroids, labels, wcss_history)`
- Run your implementation with K=2 on the preprocessed student data
- Print the final centroid coordinates
- Print the number of students in each cluster
- Plot the WCSS (within-cluster sum of squares) across iterations to show convergence

Quality (4 pts):

- Your code should handle the case where a cluster becomes empty (reinitialize or handle gracefully)
- Describe what each centroid represents in terms of the original features (e.g., "Cluster 0 centroid has high hours_studied and high attendance — these appear to be strong students")
- Run K-Means multiple times with different random seeds and discuss whether results are consistent

Task 2.2 — sklearn Verification (5 pts: C3 + Q2)

Completion (3 pts):

- Train `sklearn.cluster.KMeans` with K=2 on the same data
- Compare cluster assignments with your NumPy implementation (how many students are assigned differently?)
- Print sklearn centroids and compare with your centroids

Quality (2 pts):

Quality (2 pts):

- Explain why small differences may exist between your implementation and sklearn (hint: sklearn uses `n_init=10` by default)

Task 2.3 — Elbow Method (8 pts: C5 + Q3)

Completion (5 pts):

- Run K-Means (sklearn is fine) for $K = 1$ through 8
- Record the WCSS (inertia) for each K
- Plot the Elbow curve (K vs. WCSS)
- Identify and state the optimal K based on the elbow point

Quality (3 pts):

- Explain why WCSS always decreases as K increases (what happens at $K = N$?)
 - Discuss: is the elbow always clear? What makes this elbow easy or hard to identify?
 - If you were a university administrator using this clustering, would $K=2$ or $K=3$ be more useful? Justify your answer with a real-world interpretation of what each cluster might represent
-

Part 3: Hierarchical Clustering (20 points)

Task 3.1 — Agglomerative Clustering and Dendrogram (10 pts: C6 + Q4)

Completion (6 pts):

- Use `scipy.cluster.hierarchy.linkage` and `dendrogram` to perform agglomerative clustering with **Ward's linkage**
- Plot the full dendrogram
- Draw a horizontal cut line at your chosen distance threshold and state how many clusters it produces
- Use `sklearn.cluster.AgglomerativeClustering` with your chosen number of clusters and report the cluster sizes

Quality (4 pts):

- Explain how to read the dendrogram: what do the y-axis heights represent? What does a tall vertical gap suggest?
- Compare the dendrogram-suggested number of clusters with the Elbow Method result from Part 2 — do they agree?

Task 3.2 — Linkage Comparison (10 pts: C6 + Q4)

Completion (6 pts):

- Run agglomerative clustering with **four different linkage criteria**: `single`, `complete`, `average`, `ward`
- For each linkage, cut at K=2 clusters and report the cluster sizes
- Plot all four dendrograms side by side (2×2 subplot)

Quality (4 pts):

- Explain why single linkage produces very different results from Ward's linkage on this dataset (relate to the chaining effect discussed in lecture)
 - Which linkage method produces the most balanced clusters? Which produces the most imbalanced? Why?
 - Recommend which linkage is most appropriate for this student dataset and justify your choice
-

Part 4: DBSCAN (20 points)

Task 4.1 — Core, Border, and Noise Points (10 pts: C6 + Q4)

Completion (6 pts):

- Run `sklearn.cluster.DBSCAN` with an initial parameter setting (e.g., `eps=0.8`, `min_samples=5`)
- Report: number of clusters found, number of noise points, size of each cluster
- Identify and print the `student_id` values classified as noise points
- Create a 2D scatter plot (e.g., `hours_studied` vs. `sleep_hours`) with points colored by cluster assignment and noise points marked distinctly (e.g., black "x" markers)

Quality (4 pts):

- For at least 2-3 noise points, examine their feature values and explain *why* DBSCAN classified them as noise (are they truly outliers, or are they boundary students?)
- Relate the noise points to the concept of anomaly detection from lecture: which anomaly type (point, contextual, collective) do these most resemble?

Task 4.2 — Parameter Tuning (10 pts: C6 + Q4)

Completion (6 pts):

- Experiment with at least 5 different `eps` values (e.g., 0.5, 0.8, 1.0, 1.5, 2.0) while keeping `min_samples=5`
- For each `eps`, record: number of clusters, number of noise points, largest cluster size
- Create a summary table or plot showing how the results change with `eps`
- Select the best `eps` value and justify your choice

Quality (4 pts):

- Explain the tradeoff: what happens when `eps` is too small? Too large?
- Experiment with different `min_samples` values (e.g., 3, 5, 10) at your best `eps` — how does this affect results?
- Discuss: unlike K-Means, DBSCAN does not require specifying K in advance. Is this an advantage or disadvantage for this dataset?

Part 5: Method Comparison (10 points: C6 + Q4)

Completion (6 pts):

- Create a comparison table summarizing results from all three methods:

Method	# Clusters	Noise Points	Largest Cluster	Smallest Cluster
K-Means (best K)	...	N/A	...	...
Hierarchical (best linkage)	...	N/A	...	...
DBSCAN (best params)	...	...	...	...

- Create a side-by-side visualization: for the same 2D feature pair, plot the clustering result from each method (3 subplots)
- State which method you think works best for this dataset and why

Quality (4 pts):

- Discuss the fundamental differences: K-Means requires K, Hierarchical provides a hierarchy, DBSCAN finds arbitrary-shaped clusters and detects noise
- For a university administrator who wants to identify at-risk students, which method would you recommend? Why?

would you recommend: why:

Part 6: Cluster vs. Ground Truth Comparison (10 points: C6 + Q4)

Now load `p3_ground_truth.csv` and merge it with your clustering results.

Completion (6 pts):

- Load `p3_ground_truth.csv` and merge with your K-Means (K=2) cluster assignments by `student_id`
- Create a cross-tabulation (confusion matrix style) showing how cluster labels map to pass/fail labels
- Compute the percentage of students where cluster assignment aligns with pass/fail
- Create a visualization comparing your K-Means clusters with the true pass/fail labels side by side

Quality (4 pts):

- Discuss: did K-Means "discover" the pass/fail grouping without ever seeing the labels? How well do they align?
 - Explain why the alignment is not perfect — what factors might cause a student to be in the "wrong" cluster?
 - Reflect on the power and limitations of unsupervised learning: in what scenarios would unsupervised methods be preferred over supervised methods, even when labels are available?
-

Part 7: Summary & Reflection (5 points: C3 + Q2)

Write a summary of **300–500 words** covering:

Completion (3 pts):

- Which clustering method worked best on this dataset, and why?
- What was the most surprising finding when comparing clusters to ground truth?
- How did implementing K-Means from scratch deepen your understanding compared to just using sklearn?

Quality (2 pts):

- Connect unsupervised learning to real-world applications: give one concrete example

where clustering would be more useful than classification

- If you had more time, what additional analysis would you perform?

Grading Summary

Part	Completion	Quality	Total
Part 1: Data Preprocessing	6	4	10
Part 2: K-Means Clustering	16	9	25
Part 3: Hierarchical Clustering	12	8	20
Part 4: DBSCAN	12	8	20
Part 5: Method Comparison	6	4	10
Part 6: Ground Truth Comparison	6	4	10
Part 7: Summary & Reflection	3	2	5
Total	61	39	100

Grading Philosophy

The two-tier rubric (Completion + Quality) is designed so that **a student who diligently completes all tasks can earn ~61/100 from Completion alone**. Quality points reward deeper analysis, better visualizations, and thoughtful discussion. A student who completes all tasks thoroughly and provides good analysis should expect a score in the **80-90 range**. The quality tier rewards deeper insight and cleaner execution, but completion of each task is the foundation of your score.

Code Quality Expectations

- Code must be well-organized and readable with meaningful variable names.
- Include comments explaining your logic, especially for the K-Means implementation.
- All plots must have proper titles, axis labels, and legends.
- Use Markdown cells to explain your approach before each code section.

Academic Integrity

You must be able to explain every line of your submitted code. Submissions that cannot be explained by the submitting student(s) during a follow-up review will receive a zero. All group members are expected to contribute meaningfully to the project.

AI Tool Policy

You are welcome to use AI tools (ChatGPT, GitHub Copilot, Claude, etc.) to assist with coding and debugging. We do not penalize AI-assisted code. However, the **analysis, interpretation, and written discussion must demonstrate your own understanding**. Specifically:

- All Quality-tier discussion questions must reflect your genuine reasoning about the results you obtained.
- The Summary & Reflection section (Part 7) must be written in your own words.
- During a follow-up review, you should be able to explain your analysis and why you reached your conclusions.